

Handling Missing Values in Python (Pandas)

Introduction

In real-world datasets, missing data is almost inevitable-whether due to data entry issues, sensor failures, privacy concerns, or simply non-responses. If not handled properly, missing values can skew your analysis and reduce model performance. This lesson will walk you through identifying and handling missing values using Pandas.

Learning Objectives

By the end of this lesson, students will be able to:

1. Detect missing values in a dataset using Pandas.
2. Understand the consequences of ignoring missing data.
3. Choose the right method to handle missing values.
4. Apply both deletion and imputation methods using code.
5. Understand when to use mean, median, mode, or forward/backward fill.

Concept Explanation

1. Identifying Missing Values

How do missing values appear in Pandas?

Pandas represents missing data as **NaN** (Not a Number) or **None**.

Techniques to identify missing values:

python

```
import pandas as pd
import numpy as np
```

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, np.nan, 30, np.nan, 22],
    'Gender': ['F', 'M', np.nan, 'M', 'F']
}

df = pd.DataFrame(data)

python

# Check if there are any missing values (True = missing)
print(df.isnull())

# Count missing values per column
print(df.isnull().sum())

# Total number of missing values
print(df.isnull().sum().sum())

# Percentage of missing values
print(df.isnull().mean() * 100)
```

2. Handling Missing Values

A. Dropping Missing Values

Use when:

- Missingness is random.
- The affected row/column doesn't contain critical data.
- Proportion of missing data is very small.

python

```
# Drop rows with any missing values
```

```
df_drop_rows = df.dropna()

# Drop columns with any missing values
df_drop_cols = df.dropna(axis=1)

# Drop rows only if ALL values are missing
df_drop_all = df.dropna(how='all')

# Drop rows if a threshold of non-NaNs is not met
df_thresh = df.dropna(thresh=2) # Keep rows with at least 2
non-missing values
```

B. Filling Missing Values (Imputation)

1. Constant Imputation

Use when:

- You want to replace missing values with a placeholder (e.g., 'Unknown', 0).

python

```
df_fill_constant = df.fillna({'Age': 0, 'Gender': 'Unknown'})
```

2. Mean Imputation

Use for:

- **Numerical columns**
- When data is symmetrically distributed

python

```
df_mean = df.copy()
df_mean['Age'] = df['Age'].fillna(df['Age'].mean())
```

3. Median Imputation

Use for:

- **Numerical columns**
- When data has **outliers**

python

```
df_median = df.copy()
df_median['Age'] = df['Age'].fillna(df['Age'].median())
```

4. Mode Imputation

Use for:

- **Categorical columns**
- Replaces missing values with the most frequent category

python

```
df_mode = df.copy()
df_mode['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

5. Forward and Backward Fill

Use for:

- **Time series data** or ordered observations.

python

```
# Forward Fill (Propagate previous value)
```

```
df_ffill = df.fillna(method='ffill')

# Backward Fill (Use next value)
df_bfill = df.fillna(method='bfill')
```

Common Mistakes & Misunderstandings

- **Using mean on skewed data** - may not be appropriate; use median instead.
 - **Applying forward fill blindly** - may propagate wrong values.
 - **Dropping data carelessly** - may lead to loss of important info.
 - **Not checking percentage of missingness** - some columns may be too incomplete to be useful.
-

Interactive Exercise

Exercise: Titanic Dataset (from seaborn)

python

```
import seaborn as sns
df = sns.load_dataset("titanic")

# Task 1: Identify columns with missing data
print(df.isnull().sum())

# Task 2: Drop rows where 'age' is missing
df_dropped = df.dropna(subset=['age'])

# Task 3: Fill 'age' with median
df['age'] = df['age'].fillna(df['age'].median())

# Task 4: Fill 'embarked' with mode
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
```

Student-Friendly Analogy

Imagine you're baking cookies using a recipe. If one ingredient is missing:

- **Drop the whole recipe?** Only if that ingredient is critical.
 - **Use a default (constant)?** Like substituting chocolate chips with raisins.
 - **Use average amounts?** If you usually add 2 eggs, assume 2.
 - **Look at past attempts (forward fill)?** What did you do last time?
-

Recommended Resources

- Book: *Python for Data Analysis* by Wes McKinney (Ch. 5)
 - Article: Towards Data Science – Imputation Techniques
 - Video: StatQuest – Dealing with Missing Data
-

Wrap-Up / Recap

- Always begin by **identifying** missing values with `.isnull()`.
 - Choose between **dropping** or **filling** based on context.
 - Use `.fillna()` with constants, mean, median, or mode as needed.
 - For sequences, `.ffill()` and `.bfill()` are helpful.
 - Analyze impact before and after applying any technique.
-

Quiz / Reflection Questions

1. What is the difference between `.dropna()` and `.fillna()`?
2. When is forward fill more appropriate than mean imputation?
3. What's a potential drawback of using mode imputation on categorical data?
4. What are the risks of blindly dropping missing rows?